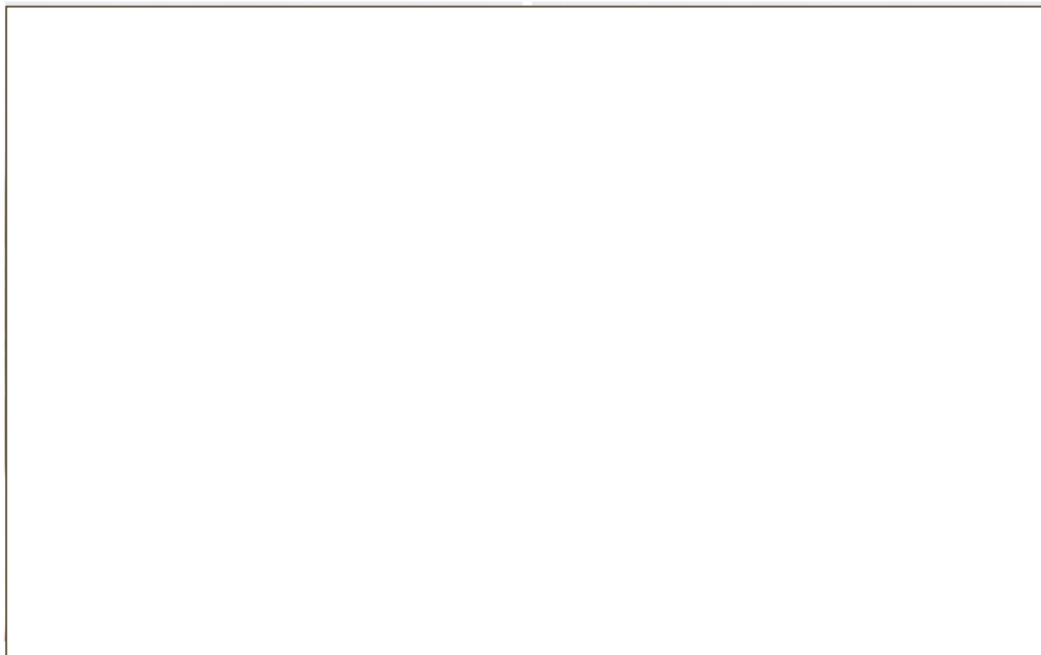
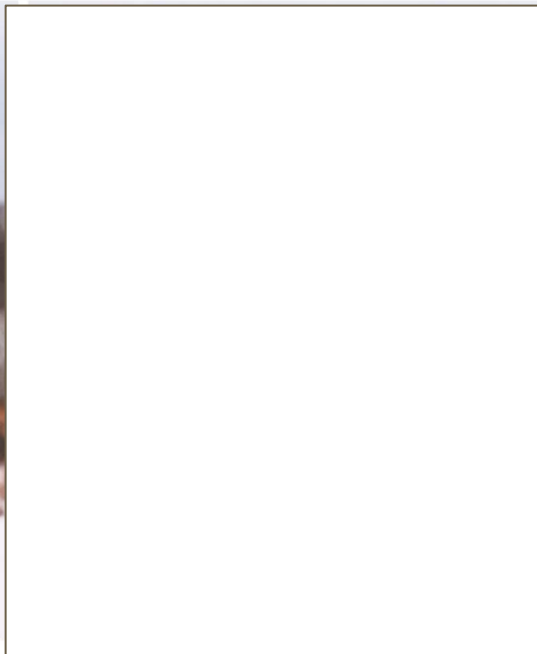


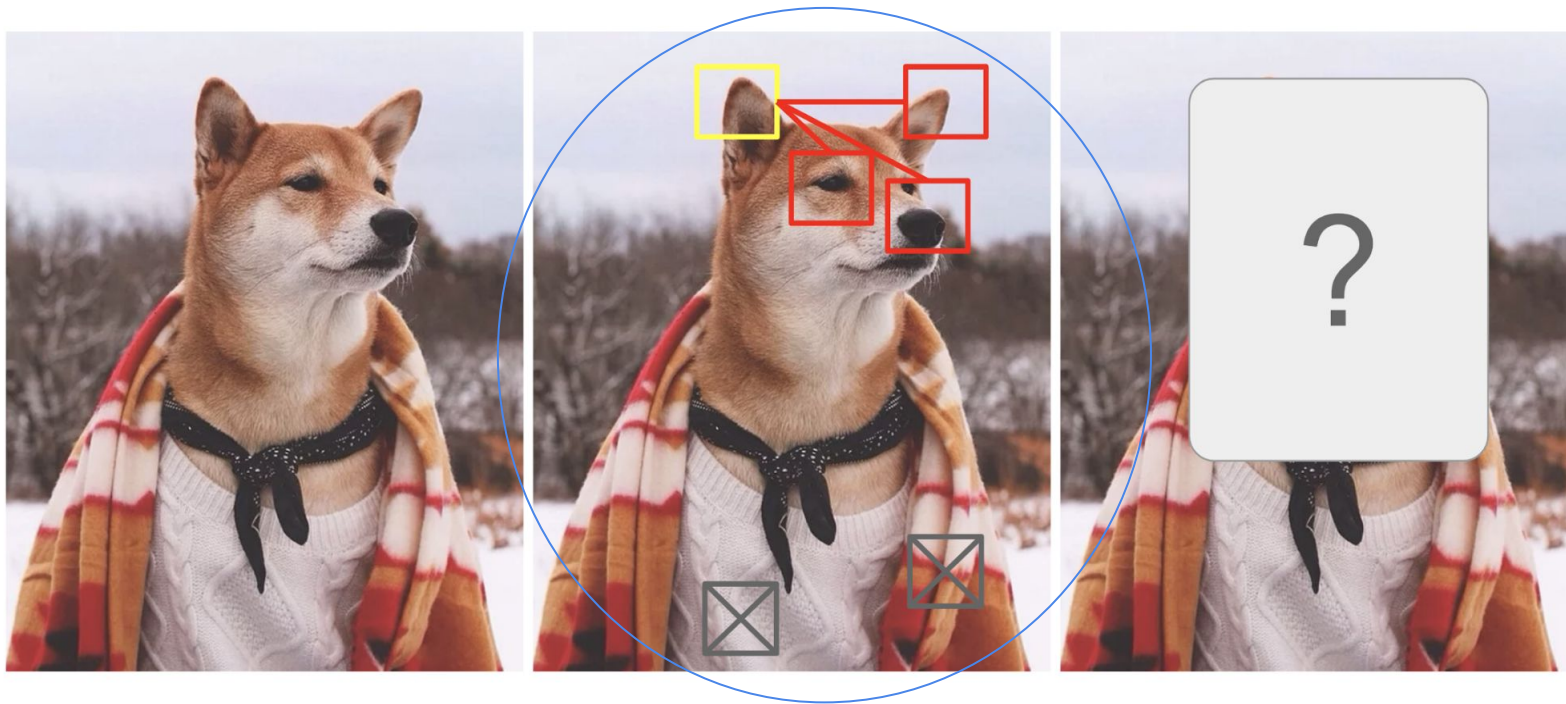
Introduction



Introduction



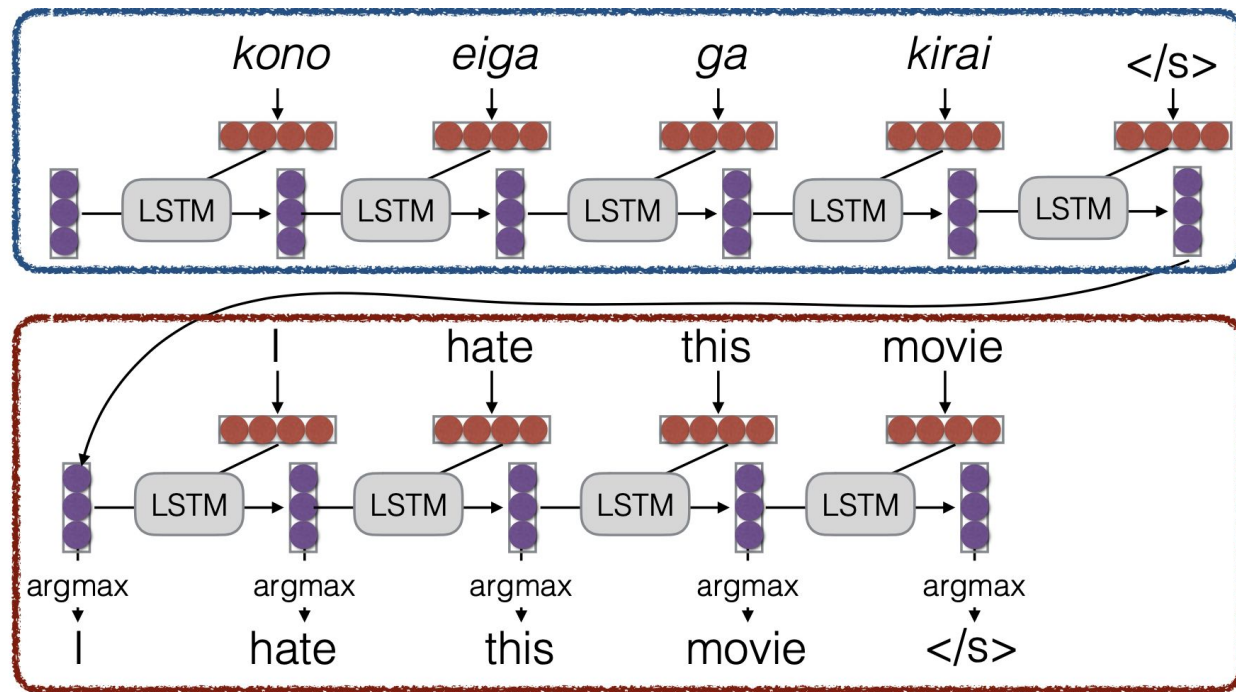
Introduction



Introduction

Encoder-decoder Models

Sutskever et al. 2014

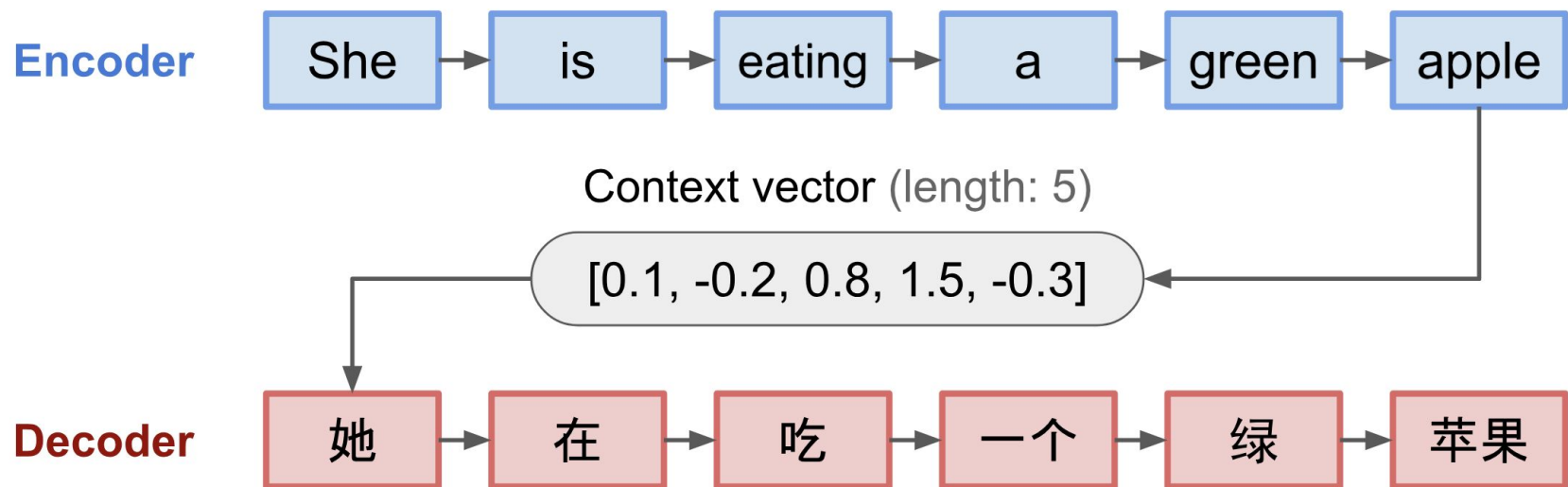


Introduction

The seq2seq model normally has an encoder-decoder architecture, composed of:

- An **encoder** processes the input sequence and compresses the information into a context vector (also known as **sentence embedding** or “**thought**” vector) of a *fixed length*. This representation is expected to be a good summary of the meaning of the *whole* source sequence.
- A **decoder** is initialized with the context vector to emit the transformed output. The early work only used the last state of the encoder network as the decoder initial state

Introduction



Introduction

- A critical and apparent disadvantage
 - fixed-length context vector design is incapability of remembering long sentences.
 - Often it has forgotten the first part once it completes processing the whole input.

Sentence Representation

Problem

“You can’t cram the meaning of a whole %&!\$ing sentence into a single \$&!*ing vector!”
— Ray Mooney

- But what if we could use multiple vectors, based on the length of the sentence.

this is an example



this is an example



Attention: Born for Translation

The attention mechanism was born (Bahdanau et al., 2015) to resolve such problems.

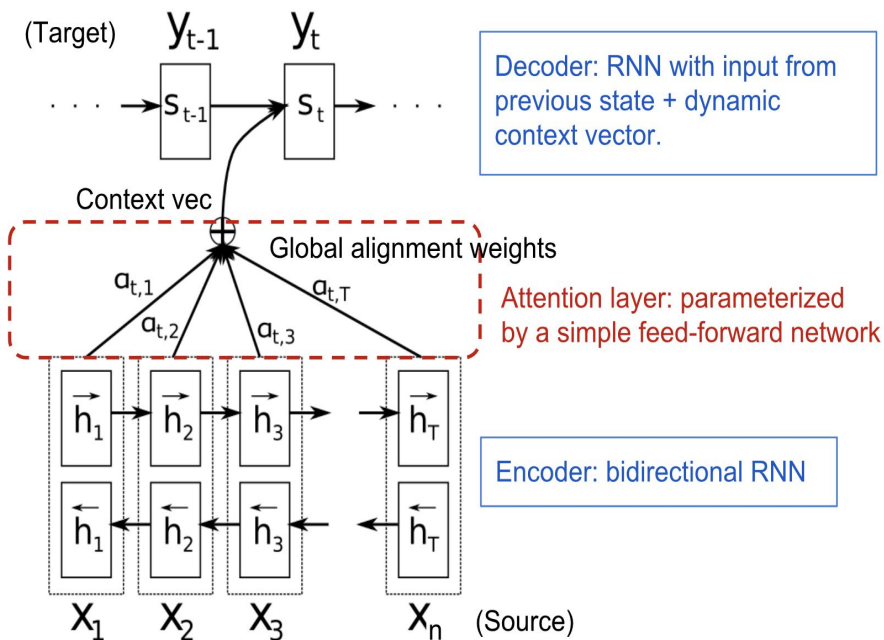
- the secret sauce invented by attention is to create shortcuts between the context vector and the entire source input.
- The weights of these shortcut connections are customizable for each output element.

While the context vector has access to the entire input sequence, **we don't need to worry about forgetting**. The alignment between the source and target is learned and controlled by the context vector.

Attention Mechanism

Context vector consumes three pieces of information:

- encoder hidden states;
- decoder hidden states;
- alignment between source and target.



Additive Attention

Attention Mechanism

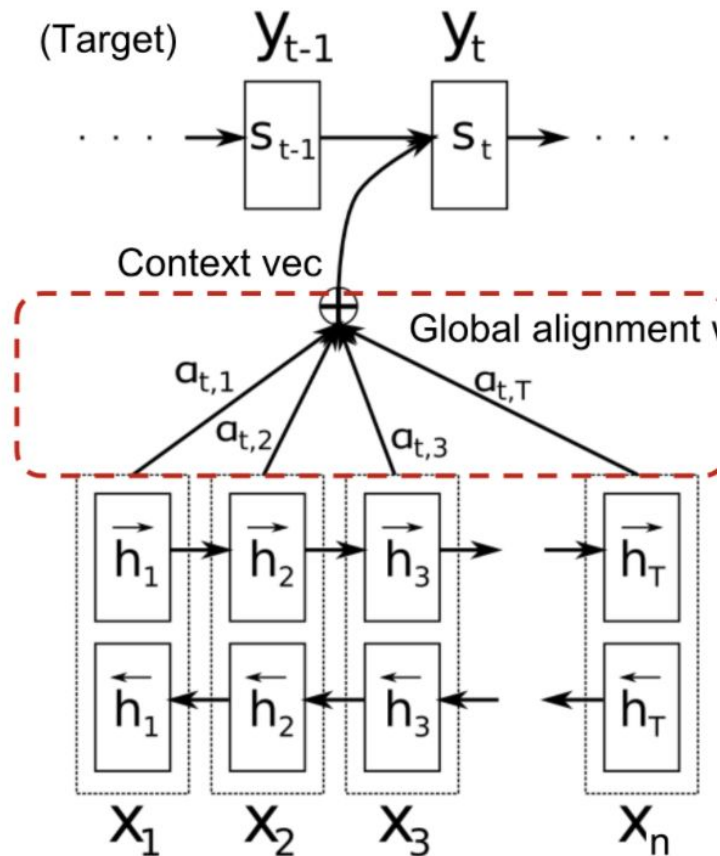
$$p(y_i|y_1, \dots, y_{i-1}, \mathbf{x}) = g(y_{i-1}, s_i, c_i)$$

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j. \quad s_i = f(s_{i-1}, y_{i-1}, c_i)$$

$$\alpha_{ij} = \frac{\exp(\text{score}(s_{t-1}, \mathbf{h}_i))}{\sum_{i'=1}^n \exp(\text{score}(s_{t-1}, \mathbf{h}_{i'}))}$$

$$\text{score}(s_t, \mathbf{h}_i) = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[s_t; \mathbf{h}_i])$$

$$\mathbf{h}_i = [\vec{\mathbf{h}}_i^\top; \overleftarrow{\mathbf{h}}_i^\top]^\top, i = 1, \dots, n$$



Attention Mechanism

● ● ● ● ● ● ● $s_i = f(s_{i-1}, y_{i-1}, c_i)$

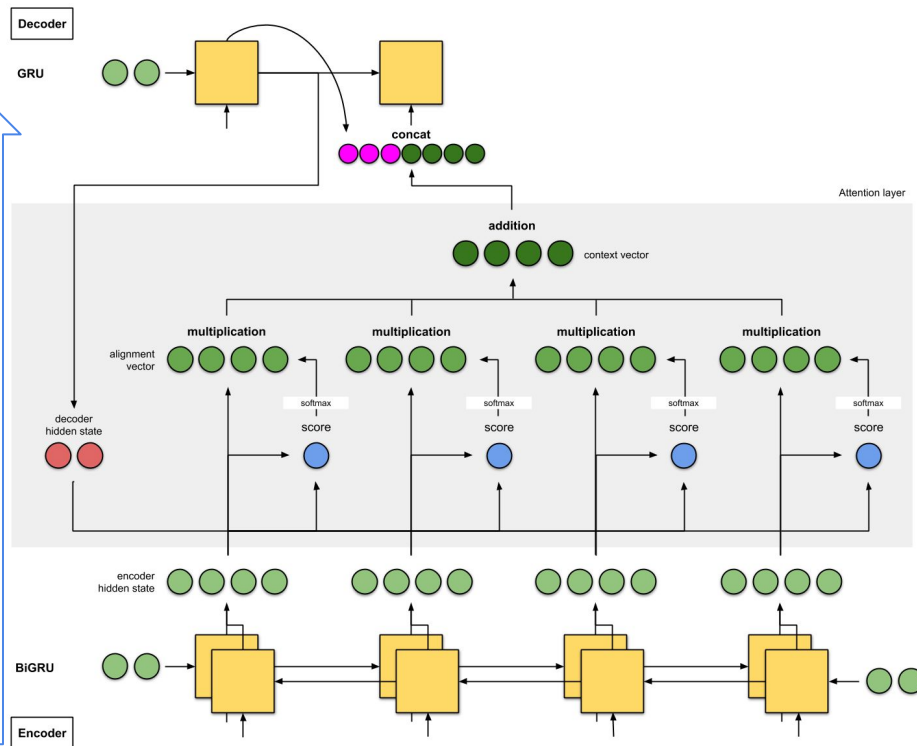
● ● ● ● $c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$

(softmax) $\alpha_{ij} = \frac{\exp(\text{score}(s_{t-1}, h_i))}{\sum_{i'=1}^n \exp(\text{score}(s_{t-1}, h_{i'}))}$



● $\text{score}(s_t, h_i) = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[s_t; h_i])$

● ● ● ● $h_i = [\vec{h}_i^\top; \overleftarrow{h}_i^\top]^\top, i = 1, \dots, n$
● ● ● ●



Attention Mechanism based on Score()

Name	Alignment score function	Citation
Content-base attention	$\text{score}(s_t, h_i) = \text{cosine}[s_t, h_i]$	Graves2014
Additive(*)	$\text{score}(s_t, h_i) = \mathbf{v}_a^\top \tanh(\mathbf{W}_a[s_t; h_i])$	Bahdanau2015
Location-Base	$\alpha_{t,i} = \text{softmax}(\mathbf{W}_a s_t)$ Note: This simplifies the softmax alignment to only depend on the target position.	Luong2015
General	$\text{score}(s_t, h_i) = s_t^\top \mathbf{W}_a h_i$ where $\mathbf{W}_a$ is a trainable weight matrix in the attention layer.	Luong2015
Dot-Product	$\text{score}(s_t, h_i) = s_t^\top h_i$	Luong2015
Scaled Dot-Product(^)	$\text{score}(s_t, h_i) = \frac{s_t^\top h_i}{\sqrt{n}}$ Note: very similar to the dot-product attention except for a scaling factor; where n is the dimension of the source hidden state.	Vaswani2017

decoder state



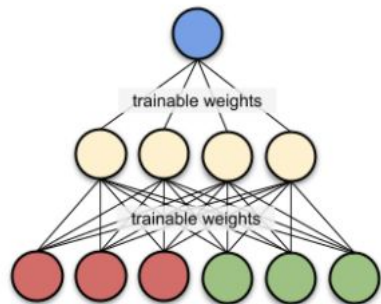
encoder state



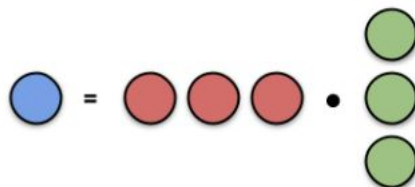
score



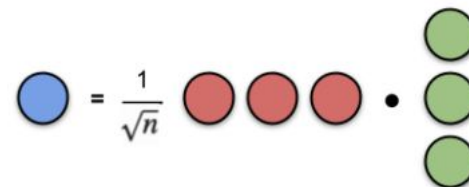
Additive / Concat



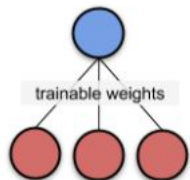
Dot product



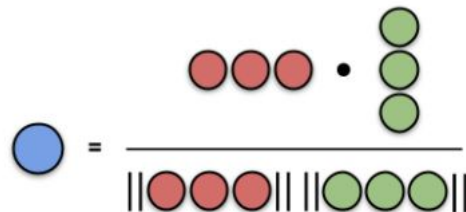
Scaled dot product



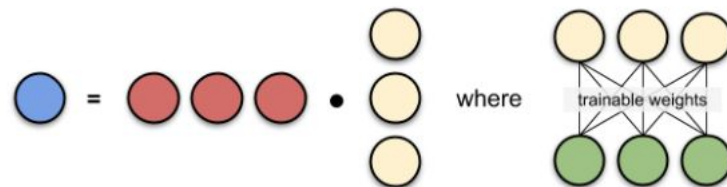
Location-based



Cosine similarity

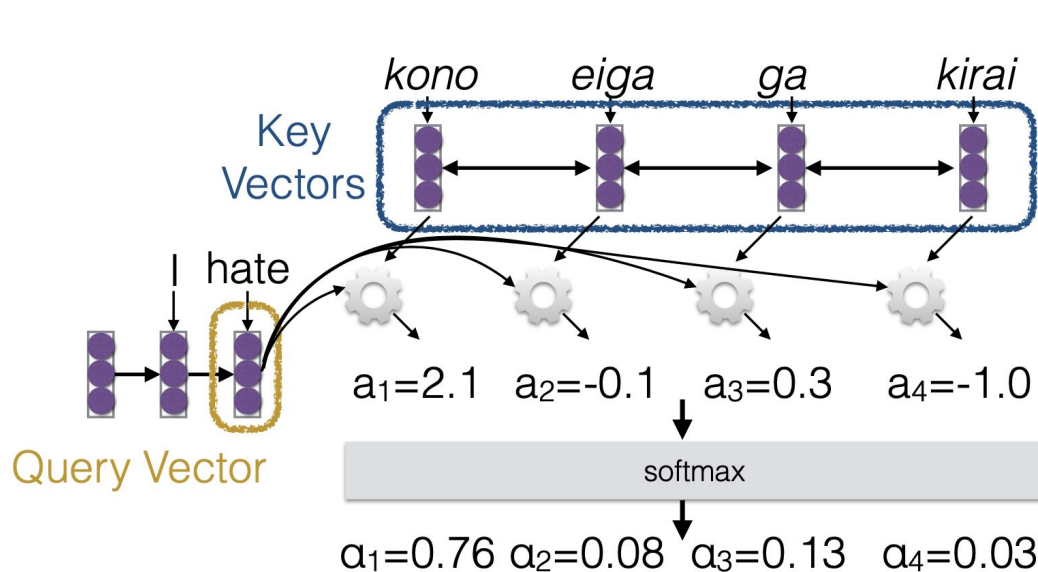


General



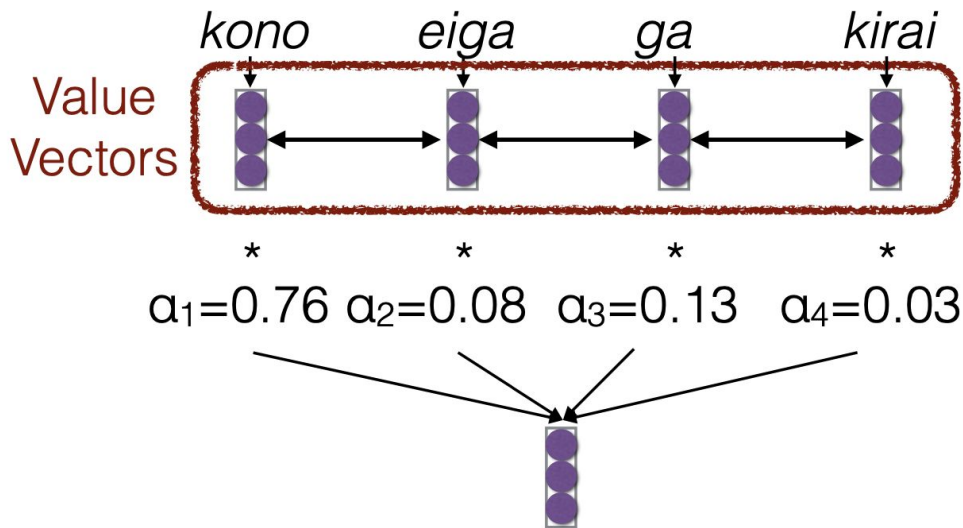
Calculating Attention (1)

- Use “query” vector (decoder state) and “key” vectors (all encoder states)
- For each query-key pair, calculate weight
- Normalize to add to one using softmax



Calculating Attention (2)

Combine together value vectors
(usually encoder states, like
key vectors) by taking the
weighted sum



Attention Mechanism Category

Name	Definition	Citation
Self-Attention(&)	Relating different positions of the same input sequence. Theoretically the self-attention can adopt any score functions above, but just replace the target sequence with the same input sequence.	Cheng2016
Global/Soft	Attending to the entire input state space.	Xu2015
Local/Hard	Attending to the part of input state space; i.e. a patch of the input image.	Xu2015; Luong2015

Hard Attention

- After calculating the attention weights, we could select the encoder state which has the **highest attention weight**, as the context, instead of taking an attention-weighted sum of states. This selection of/focus on the most relevant part of the source sequence, is called **hard attention**.

$$c_j = \operatorname{argmax}_{\alpha_j} \{h_0, h_1, \dots, h_n\}$$

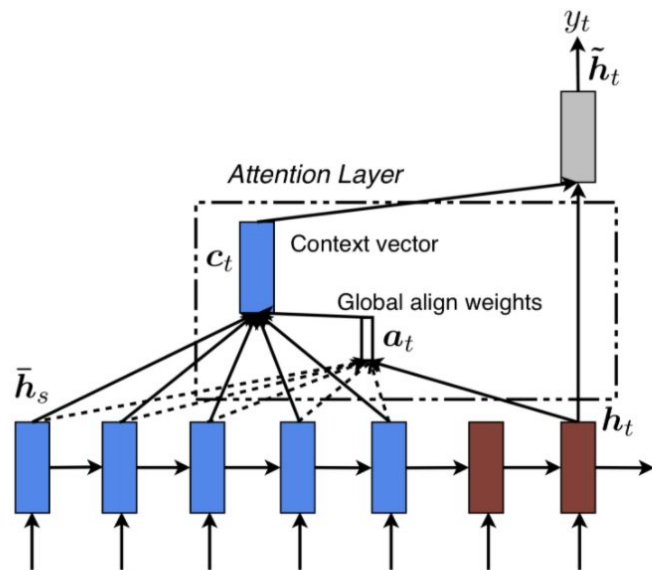
- **Soft attention** allows the decoder to consider all the states in the source sequence, weighted based on relevance.

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j.$$

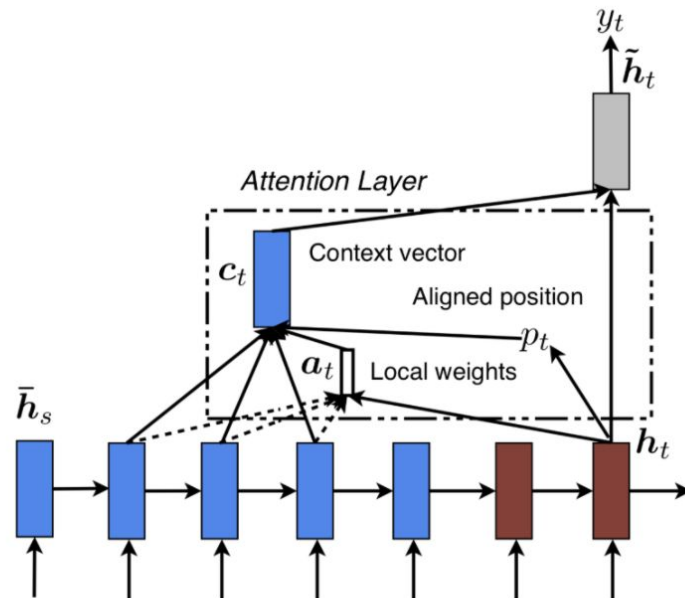
The curse of Hard Attention

- Difficult to incorporate as a component in a neural network.
- A Neural Network is fundamentally a mathematical function.
 - It is a heterogeneous composition of **multiple smaller functions (layers)**.
 - If we could represent our network as a function, then this function is expected to be **continuous**, and hence, **differentiable**.
 - Its differentiability is essential for calculating the **error gradients, backpropagate gradients, and update parameters**.
- For hard attention, the operation used in selecting the most relevant encoder state, is **NOT a continuous function** and hence, **NOT differentiable**.

Local and Global Attention



Global Attention Model



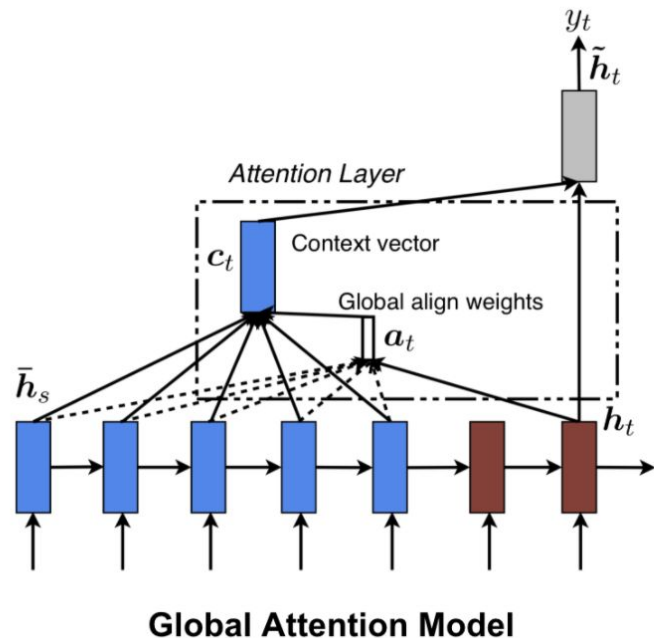
Local Attention Model

Global Attention

- All the encoder states are considered while calculating attention weights. Attention is a probability distribution across all the encoder states, using which we compose the states into a context.

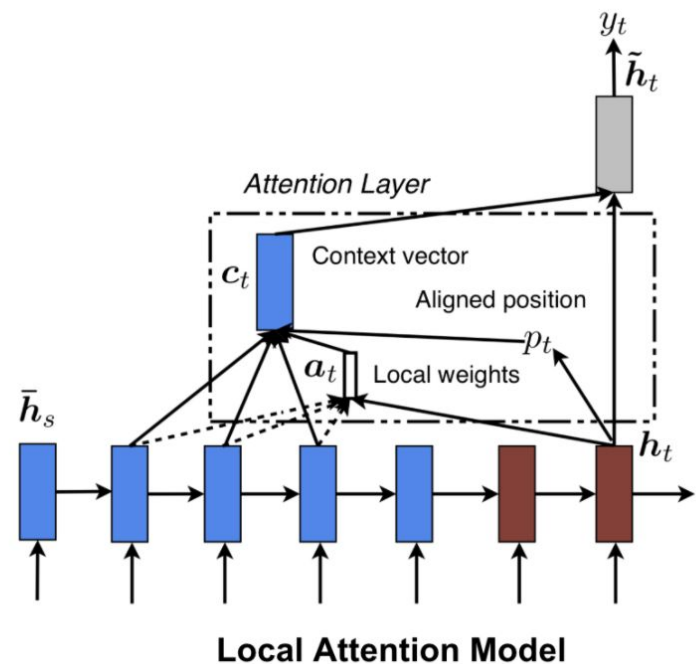
$$c_j = \sum_{i=0}^n \alpha_{ji} h_i$$

Since we consider all the encoder states, we call this configuration **global**



Local Attention

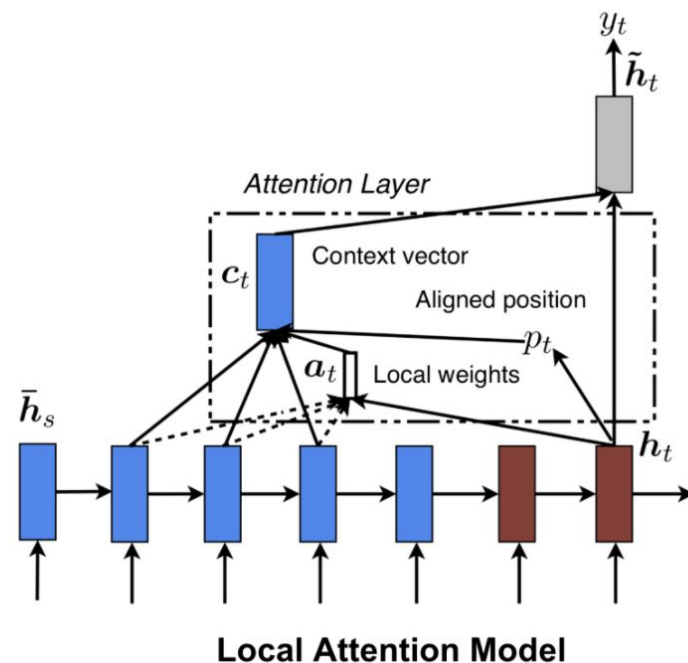
- Allow the decoder **to peak at a small segment** of the source sequence, so that the decoder could selectively focus on the tokens in that small segment.
 - *But which segment to select?*



Local Attention

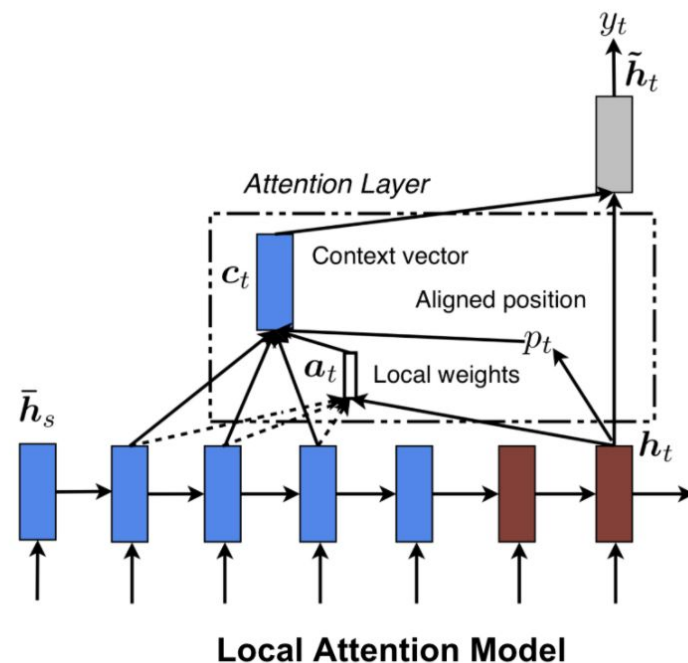
- There are two levels of selection/search here.
 - First we need to **select** which segment to pay attention to,
 - then select which tokens in the segment to pay attention to.
 - *But how long should the segment be?*
 - We select (empirically) a hyperparameter D

$$\alpha_j \in \mathbb{R}^{2D+1}$$



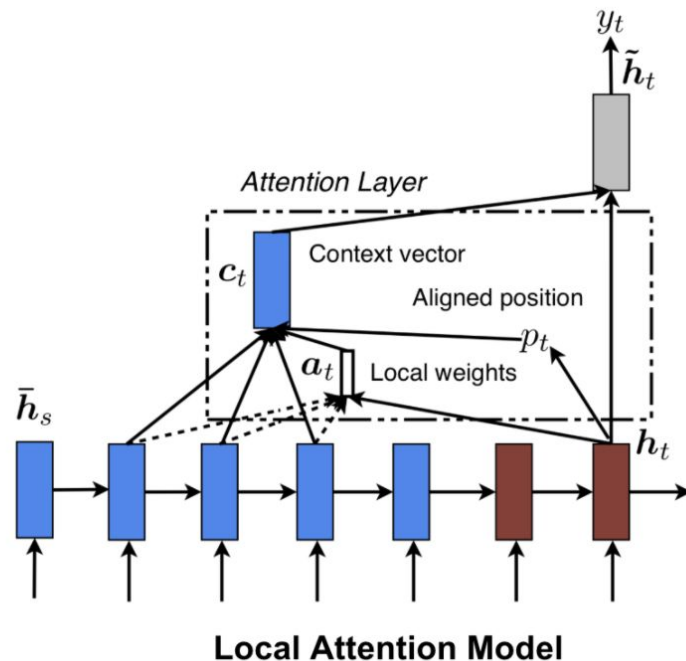
Local Attention

-
- The number of attention weights is now **fixed and dependent on D** , instead of being dependent of the source sequence length, which is variable in nature.



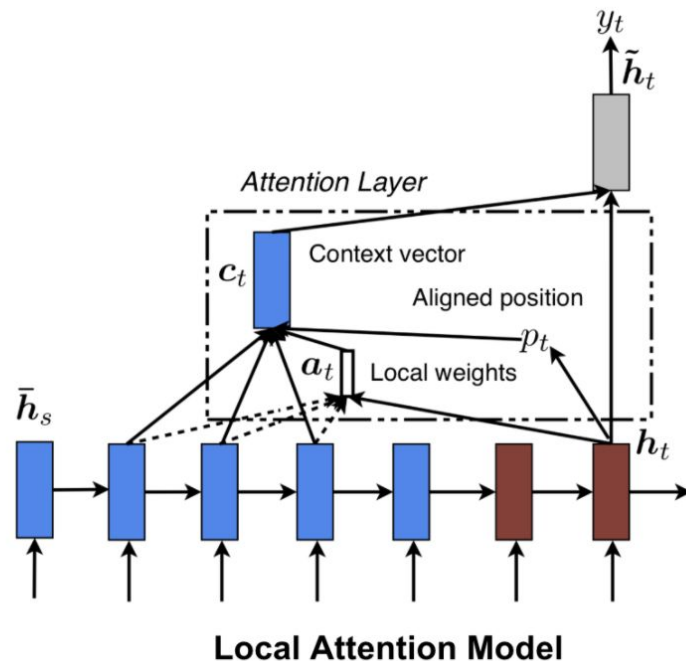
Local Attention

- In order to select a window of interest, we need to correctly place the **center of the window**. i.e., we need to select **a token** (location) in the source sequence which will be **the center of attention** – the center of the window.



Local Attention

- What is the problem here? We have run into *hard attention* again. If we use an $\mathbf{p}_t$ to select the most interesting token (location) in the source sequence, the network becomes non-differentiable. Luong et. al. 2015 proposed a clever hack to circumvent this issue.
- We first calculate *Predictive Alignment* ($\mathbf{p}_t$), by reducing the current decoder state $\mathbf{s}_t$ to a “rough location” on the source sequence.



Local Attention

- S = Source sentence length;
- $\mathbf{W}_p$ and $\mathbf{v}_p$ are the model parameters which will be learned to predict positions.
- $p_t \in [0, S]$

$$p_t = S \cdot \text{sigmoid}(\mathbf{v}_p^\top \tanh(\mathbf{W}_p \mathbf{h}_t)).$$

Local Attention

- To favor alignment points near p_t , we place a **Gaussian distribution** – a function that is differentiable – continuous throughout $(-\infty, +\infty)$. Gaussian function is continuous and can be centered around p_t
- The standard deviation is empirically set as $\sigma = D/2$

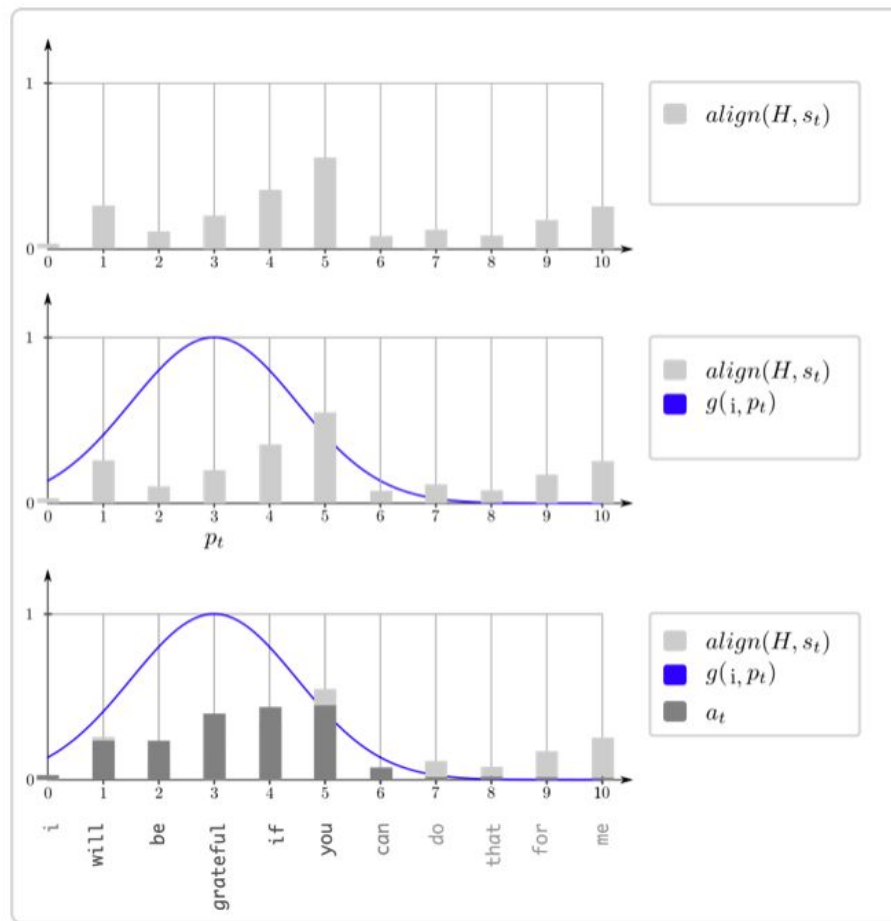
$$g(i, p_t) = \exp\left(-\frac{(i - p_t)^2}{2\sigma^2}\right)$$

$$\sigma = \frac{D}{2}$$

$$a_t = \text{align}(s_t, h_i) \cdot g(i, p_t)$$

$$a_t(s) = \text{align}(\mathbf{h}_t, \bar{\mathbf{h}}_s) \exp\left(-\frac{(s - p_t)^2}{2\sigma^2}\right)$$

- The **Gaussian function** applied over alignment, truncates the attention values in such a way that the tokens near p_t are favored over the rest of the tokens in the source sequence.
- Local attention fine-tunes the global alignment by shifting the focus near p_t .



Self Attention

- Self-attention is about attending to words within the sequence, such as within the encoder or decoder.
- By seeing how one word attends to other words in the sequence, we're able to capture syntactic structures or dependencies.

The animal didn't cross the street because it was too tired.

The animal didn't cross the street because it was too wide.

Self Attention

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

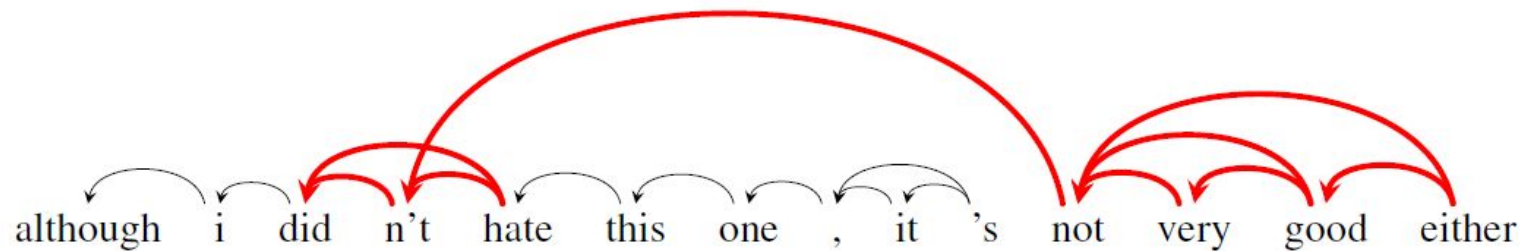
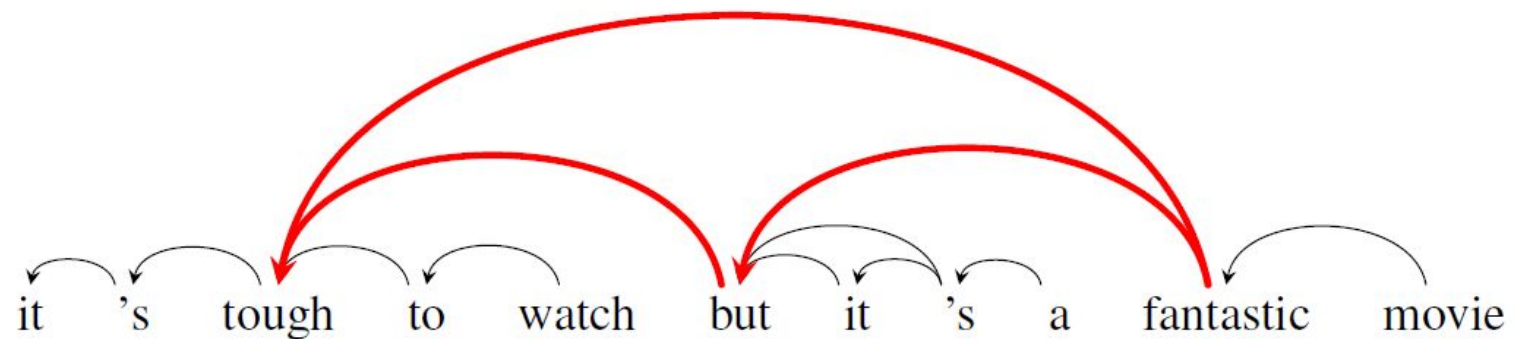
The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

The FBI is chasing a criminal on the run .

Self Attention



Self Attention

We will discuss more detailed during Transformer Model

Modeling Attentions: Multi-head Attention

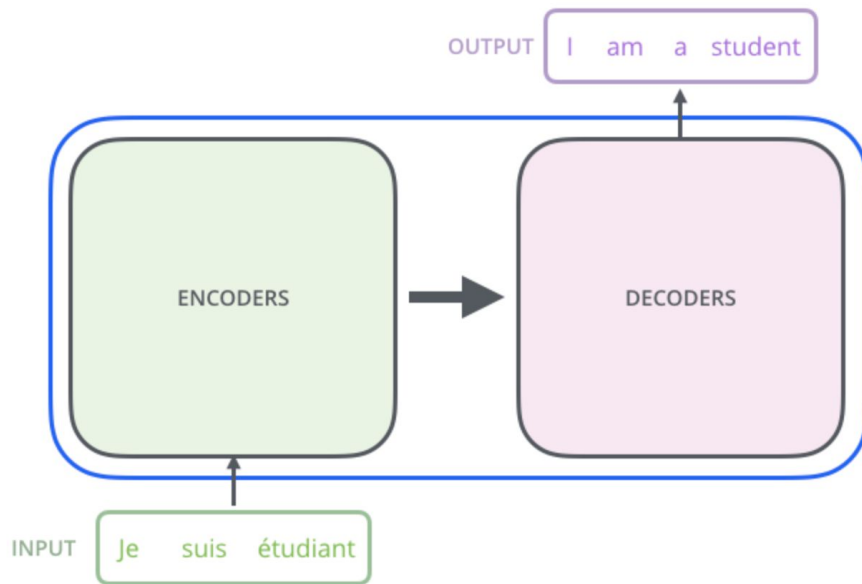
We will discuss more detailed during Transformer Model

Transformer Model



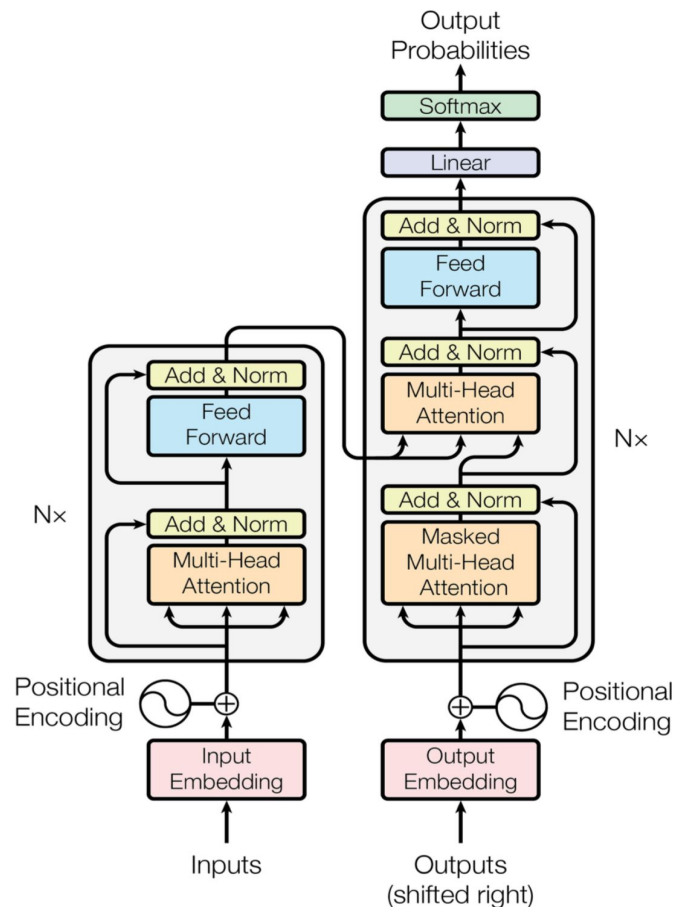
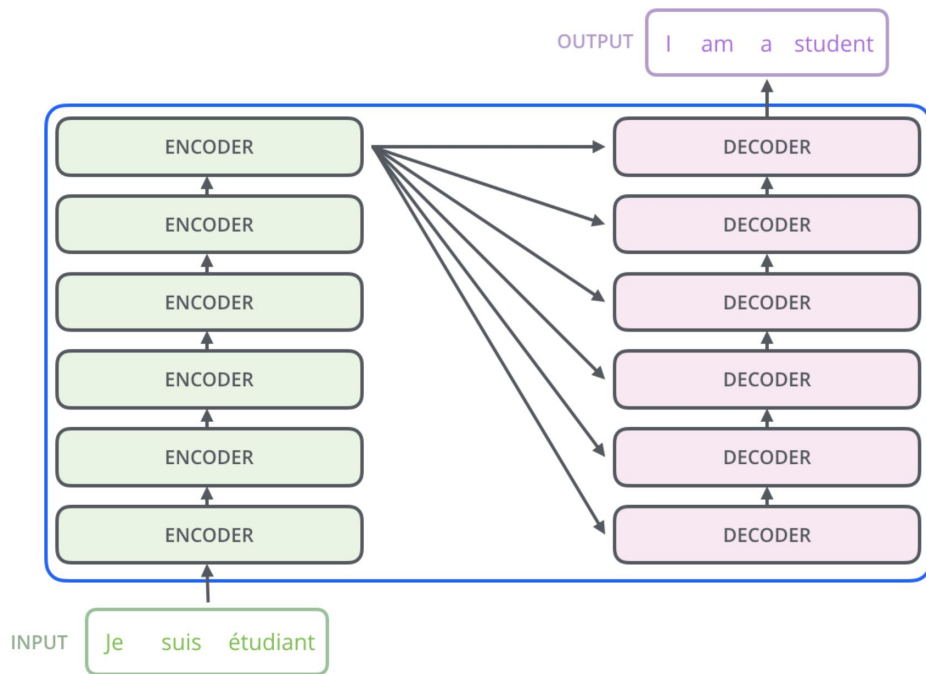
<https://jalammar.github.io/illustrated-transformer/>

Transformer Model



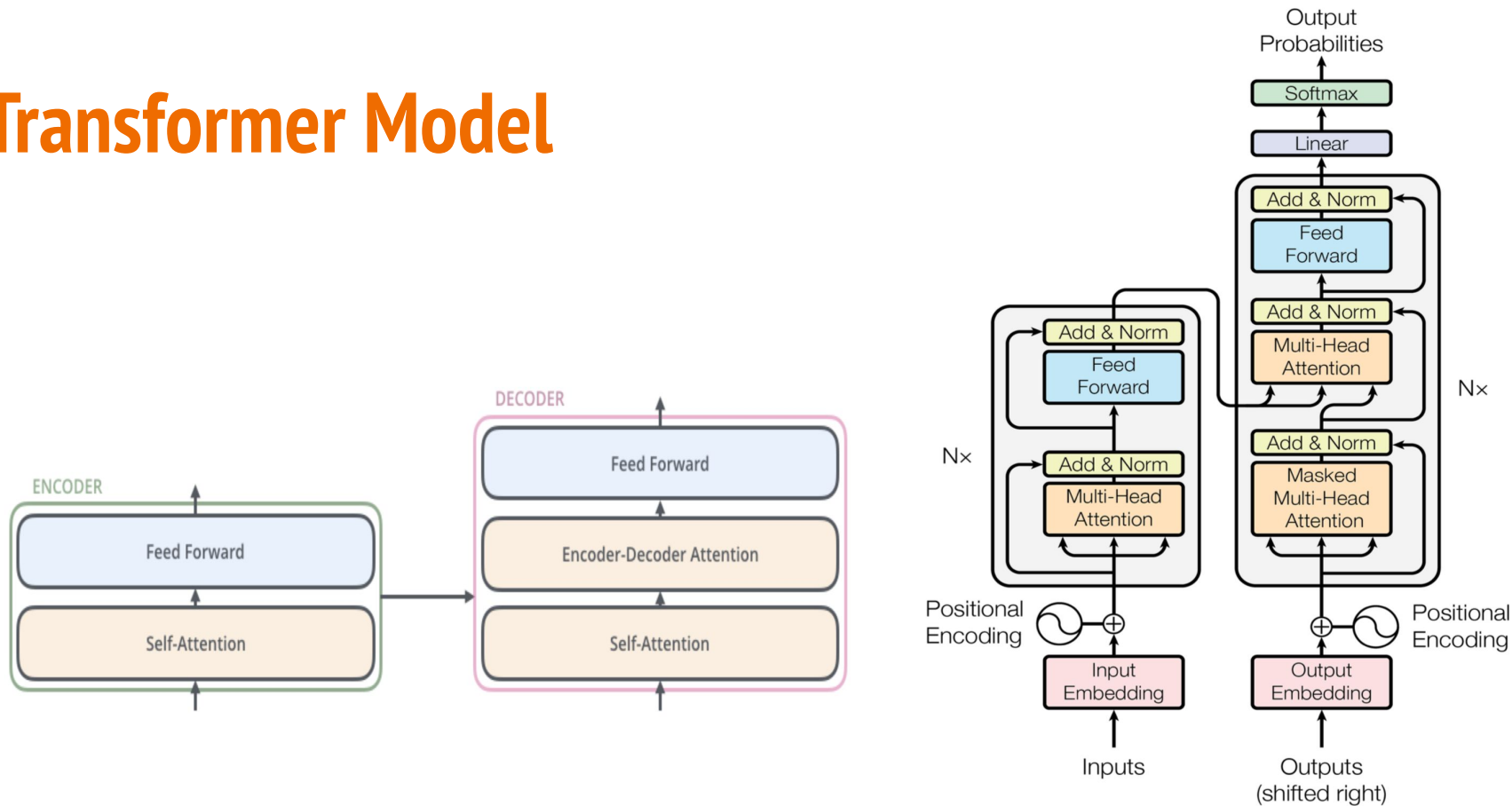
<https://jalammar.github.io/illustrated-transformer/>

Transformer Model



<https://jalammar.github.io/illustrated-transformer/>

Transformer Model



<https://jalammar.github.io/illustrated-transformer/>

Self-Attention

Self-attention is a sequence-to-sequence operation: a sequence of vectors goes in, and a sequence of vectors comes out.

Let's call the input vectors $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t$ and the corresponding output vectors $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_t$.

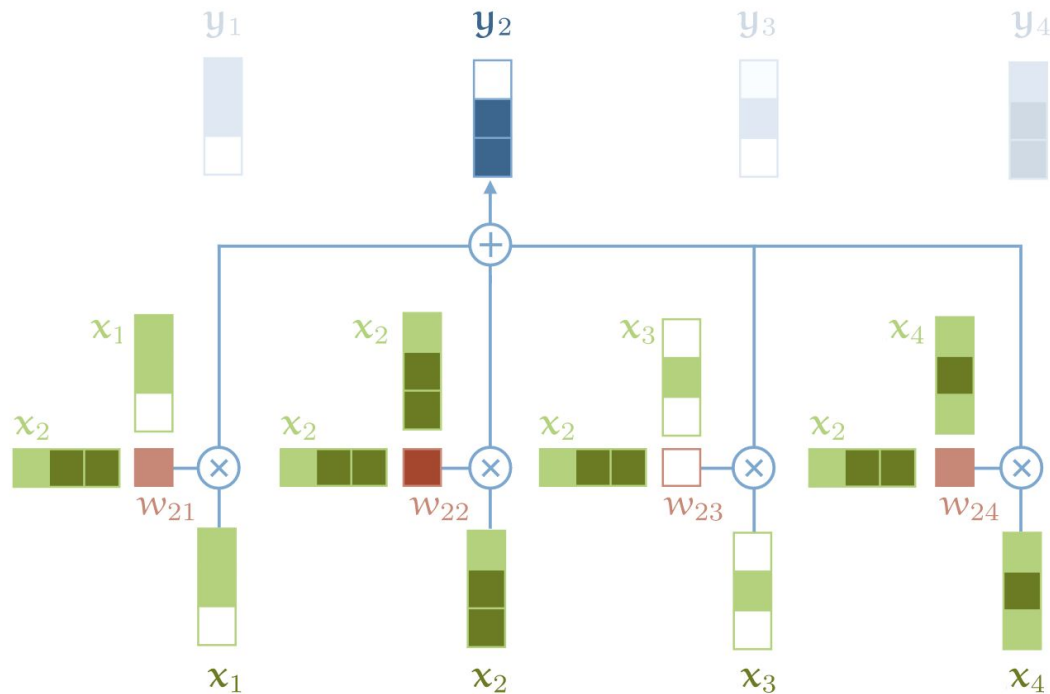
The vectors all have dimension k

$$\mathbf{y}_i = \sum_j w_{ij} \mathbf{x}_j$$

$$w_{ij} = \exp w'_{ij} / \sum_j \exp w'_{ij}$$

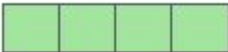
$$W'_{ij} = \mathbf{x}_i^T \mathbf{x}_j$$

Self-Attention

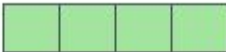


Self-Attention

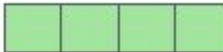
- Each word is embedded into a vector of size 512. We'll represent those vectors with these simple boxes.
- The size of this list is hyperparameter we can set – basically it would be the length of the longest sentence in our training dataset.
- 1st encoder layer that would be the word embeddings, but in other encoders, it would be the output of the encoder that's directly connected on it.

x_1 

Je

x_2 

suis

x_3 

étudiant

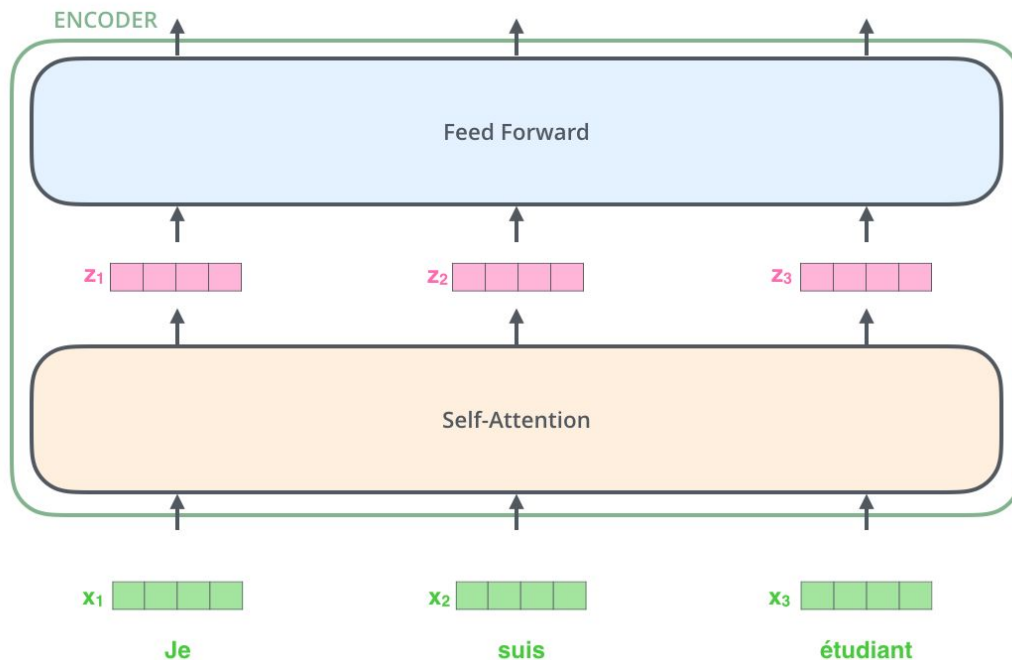
Self-Attention: Example of Input embeddings

For the word that's the 4th entry in the vocabulary, its vector is (10,12,19).

$$[0 \quad 0 \quad 0 \quad 1 \quad 0] \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = [10 \quad 12 \quad 19]$$

Self-Attention

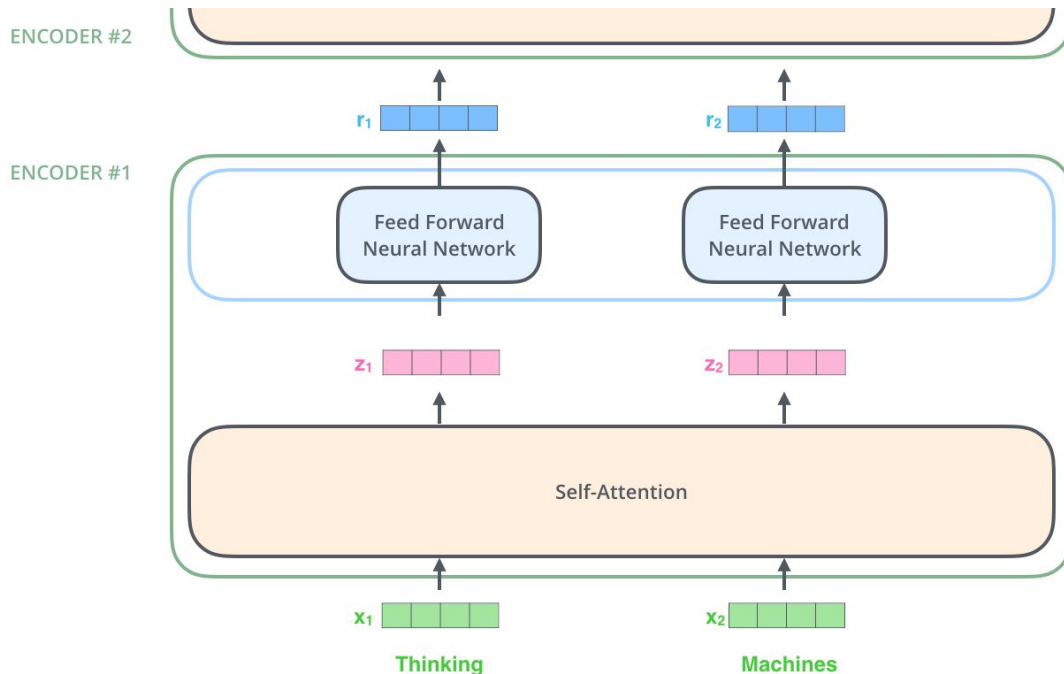
- the word in each position flows through its own path in the encoder
- There are dependencies between these paths in the self-attention layer.
- The feed-forward layer does not have those dependencies, however, and thus the various paths can be executed in parallel while flowing through the feed-forward layer.



Self-Attention

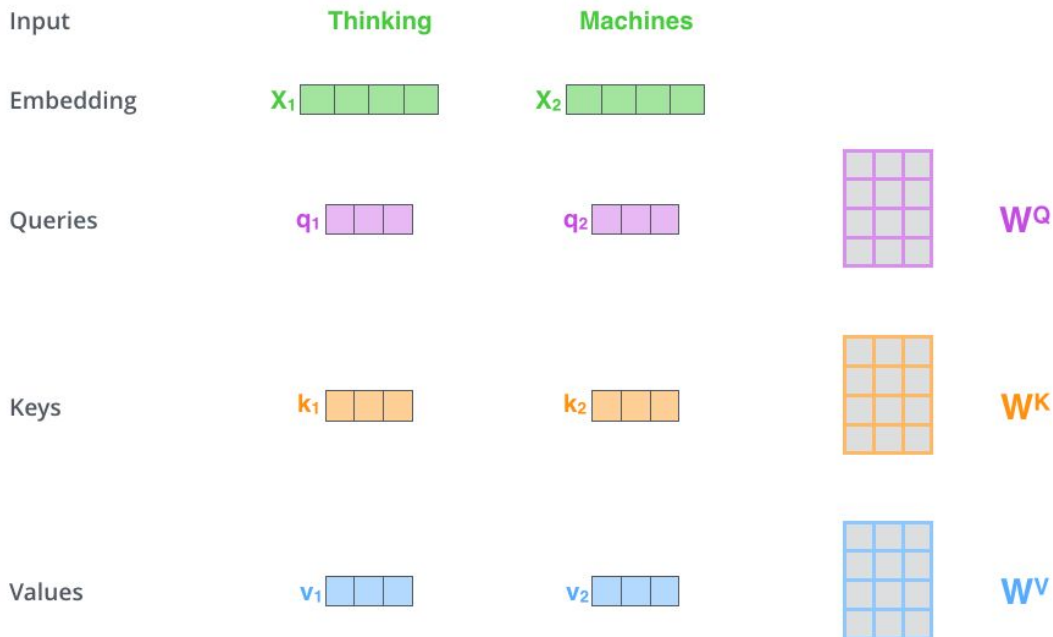
An encoder receives a list of vectors as input.

It processes this list by passing these vectors into a '**self-attention**' layer, then into a **feed-forward neural network**, then sends out the output upwards to the **next encoder**.



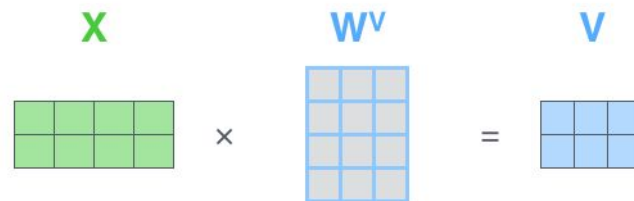
Self-Attention

- Vectors q , k & v are smaller in dimension than the embedding vector.
- $\text{Dim} = 64$, while the embedding & encoder input/output vectors have 512.
- Choice to make the computation of multi headed attention (mostly) constant.

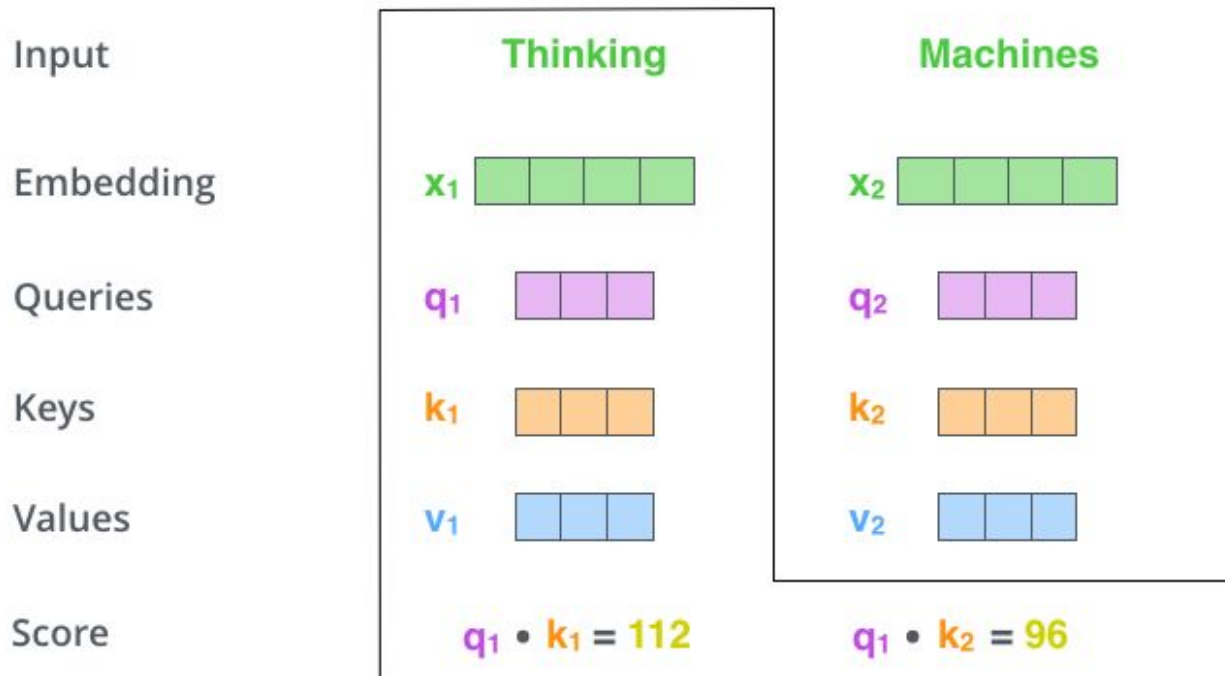


Self-Attention

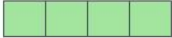
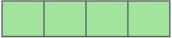
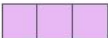
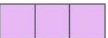
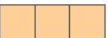
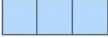
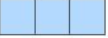
- $q_1 = (x_1 W^Q)$,
- $k_1 = (x_1 W^K)$,
- $v_1 = (x_1 W^V)$



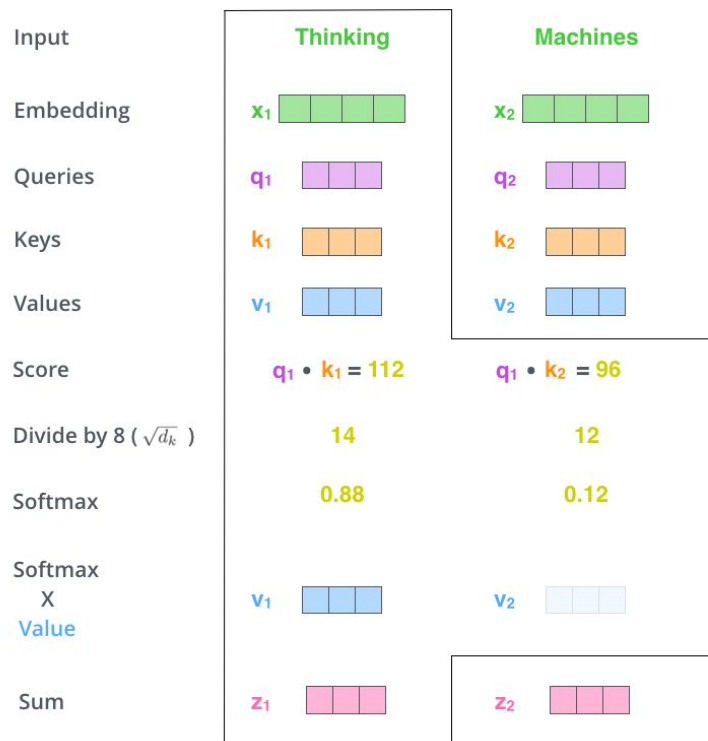
Self-Attention



Self-Attention

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Self-Attention



Self-Attention: Scale Dot Attention

$$\text{softmax}\left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix}\right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

=

Z

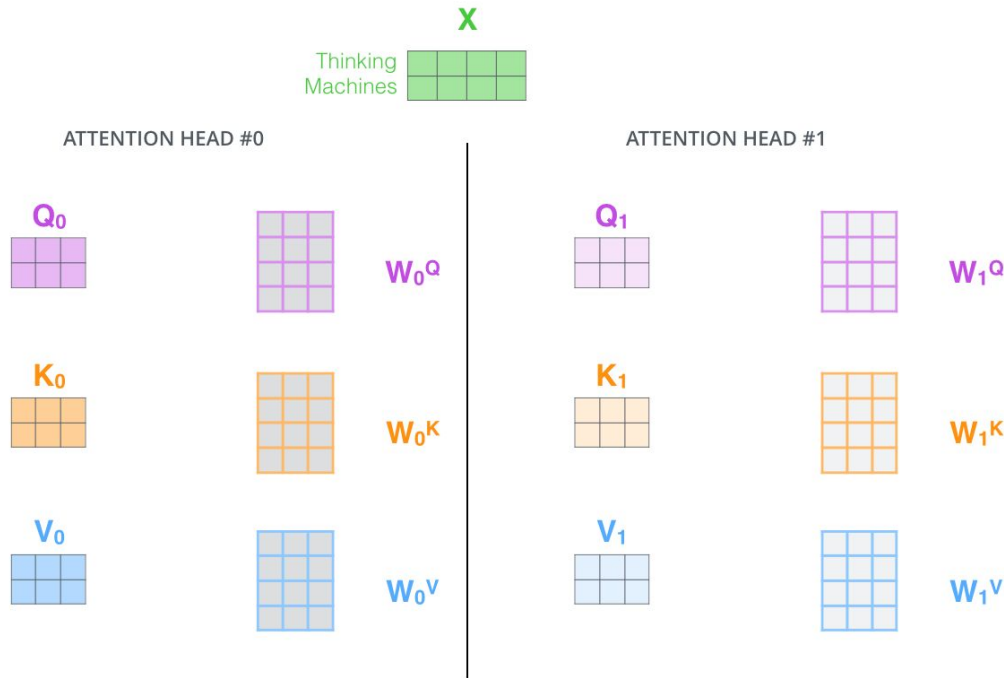
$\begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array}$

Multi-head Attention (1)

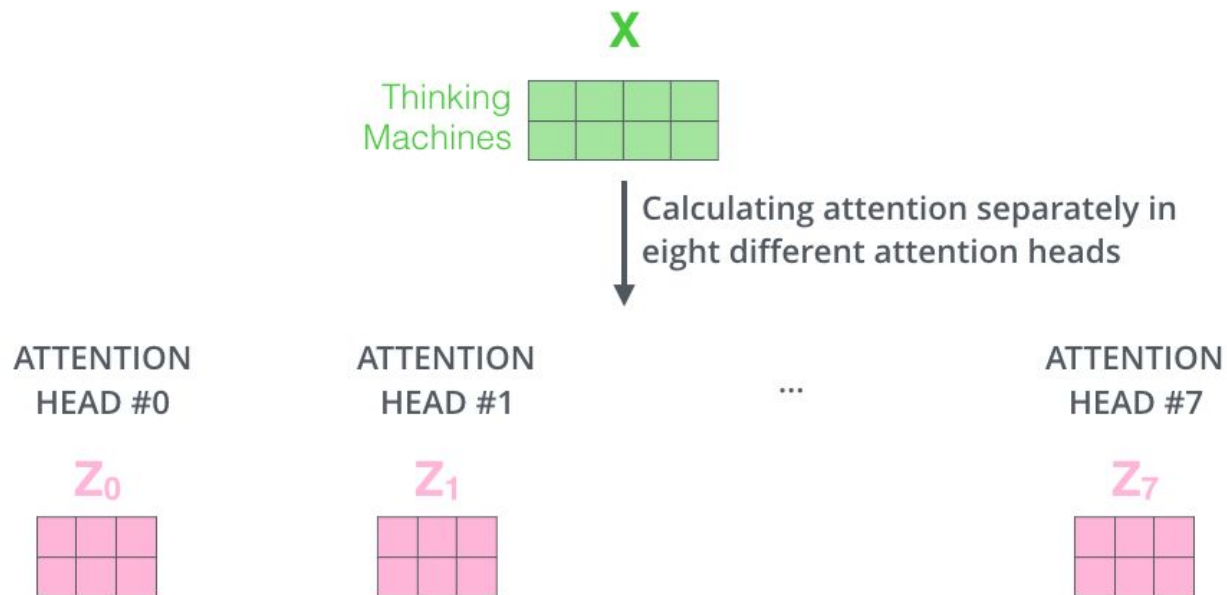
- WHY

- It expands the model's ability to focus on different positions
- It gives the attention layer multiple “representation subspaces”.

W_h^Q , W_h^K , W_h^V are randomly initialized and trainable



Multi-head Attention (2)



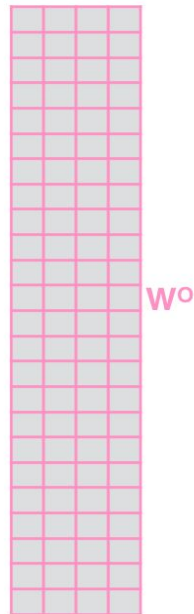
Multi-head Attention (3)

1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



Multi-head Attention (4): Summerize

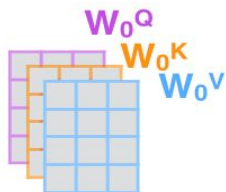
1) This is our input sentence*

Thinking
Machines

2) We embed each word*



3) Split into 8 heads.
We multiply X or R with weight matrices



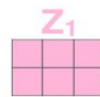
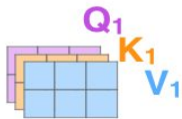
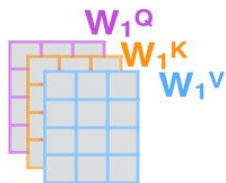
4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



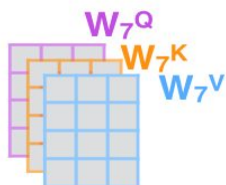
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



...

...

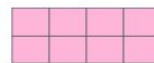
...



W^O

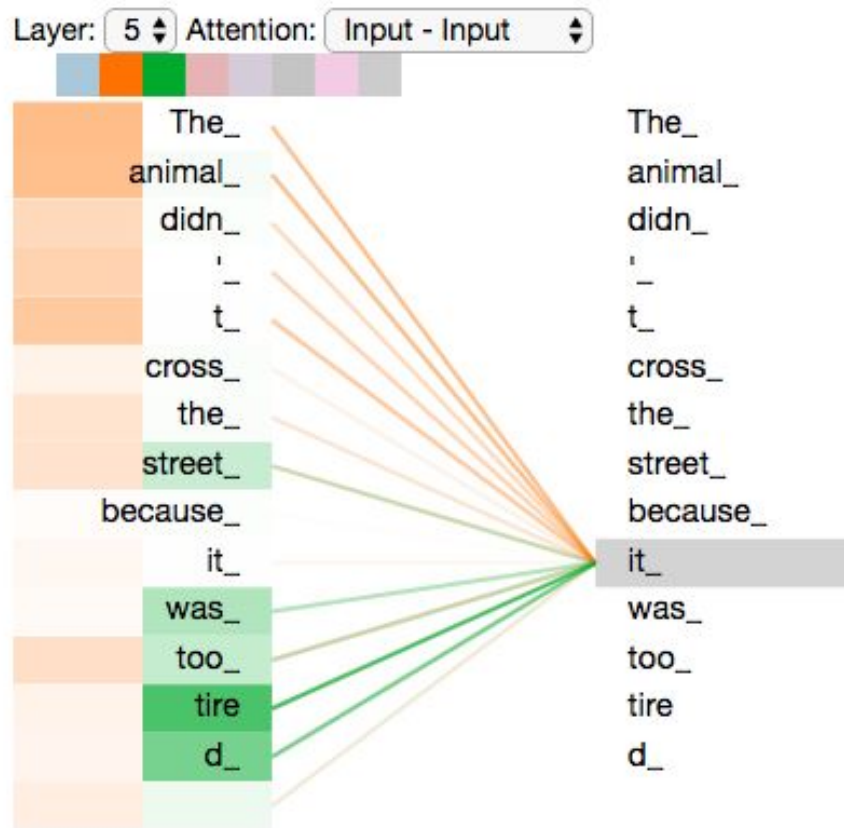


Z

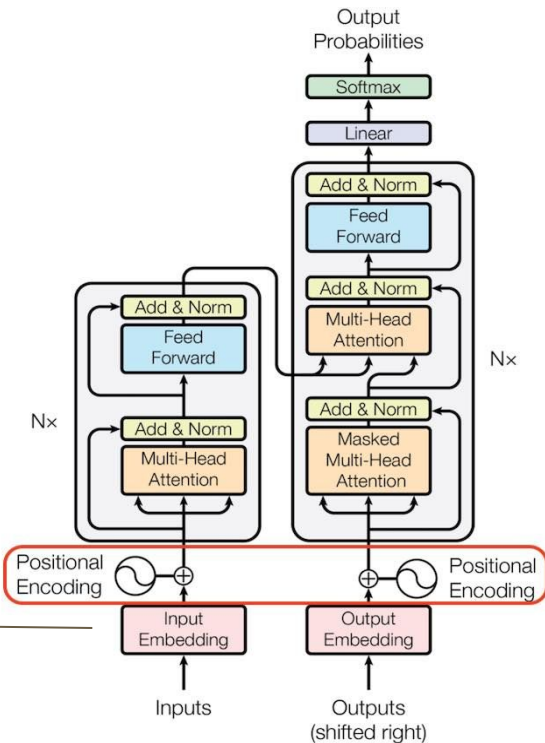
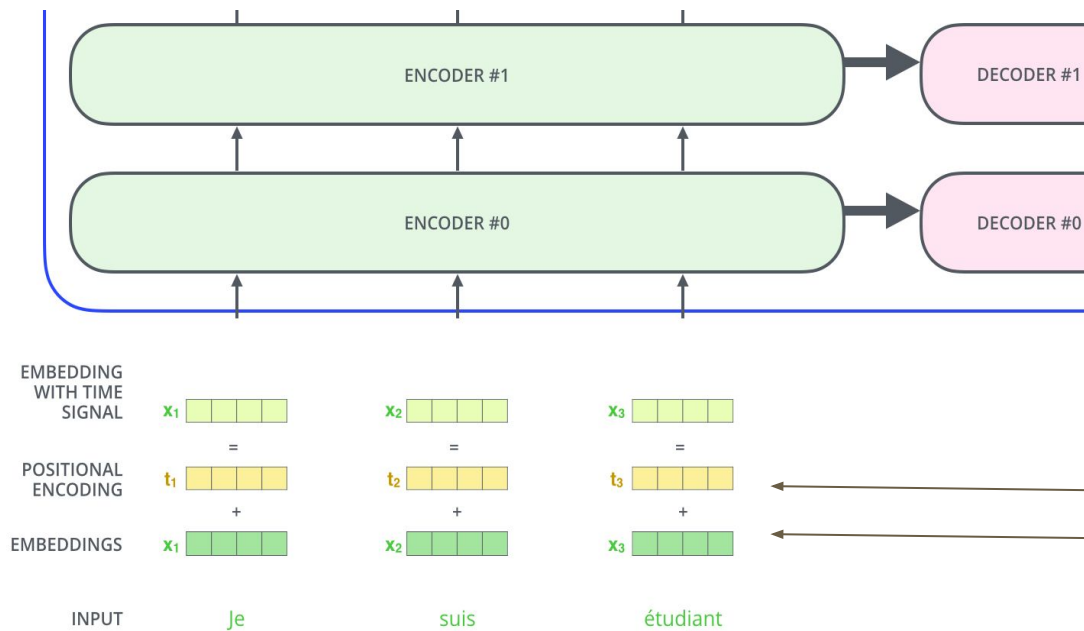


Self-Attention

As we encode the word "it", one attention head is focusing most on "the animal", while another is focusing on "tired" -- in a sense, the model's representation of the word "it" bakes in some of the representation of both "animal" and "tired".



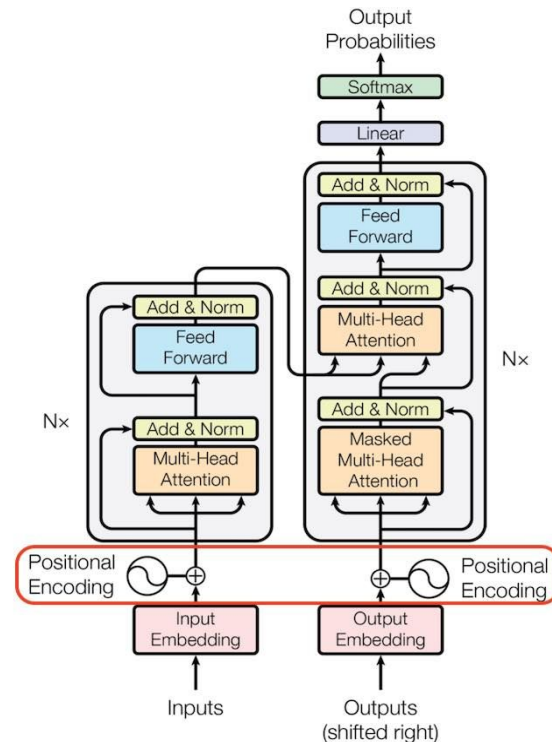
Positional Encoding



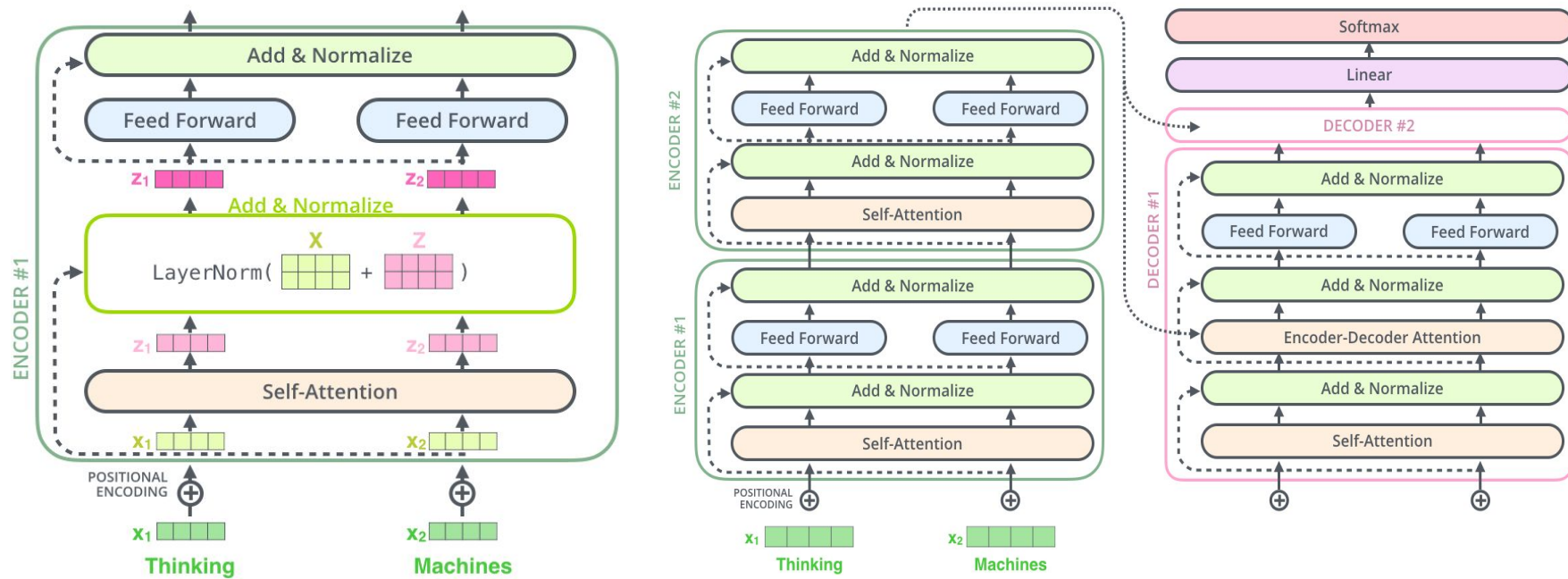
Positional Encoding

$$\vec{p}_t = \begin{bmatrix} \sin(\omega_1 \cdot t) \\ \cos(\omega_1 \cdot t) \\ \sin(\omega_2 \cdot t) \\ \cos(\omega_2 \cdot t) \\ \vdots \\ \sin(\omega_{d/2} \cdot t) \\ \cos(\omega_{d/2} \cdot t) \end{bmatrix}_{d \times 1}$$

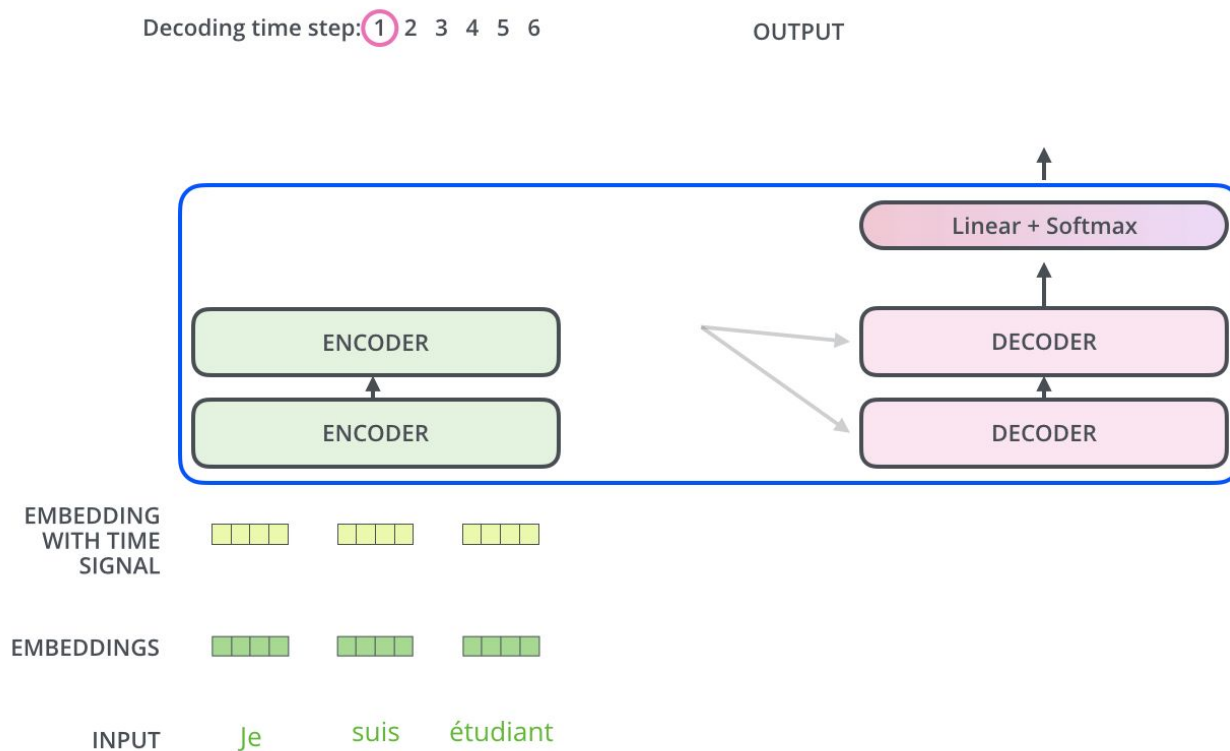
$$\omega_k = \frac{1}{10000^{2k/d}}$$



Residual

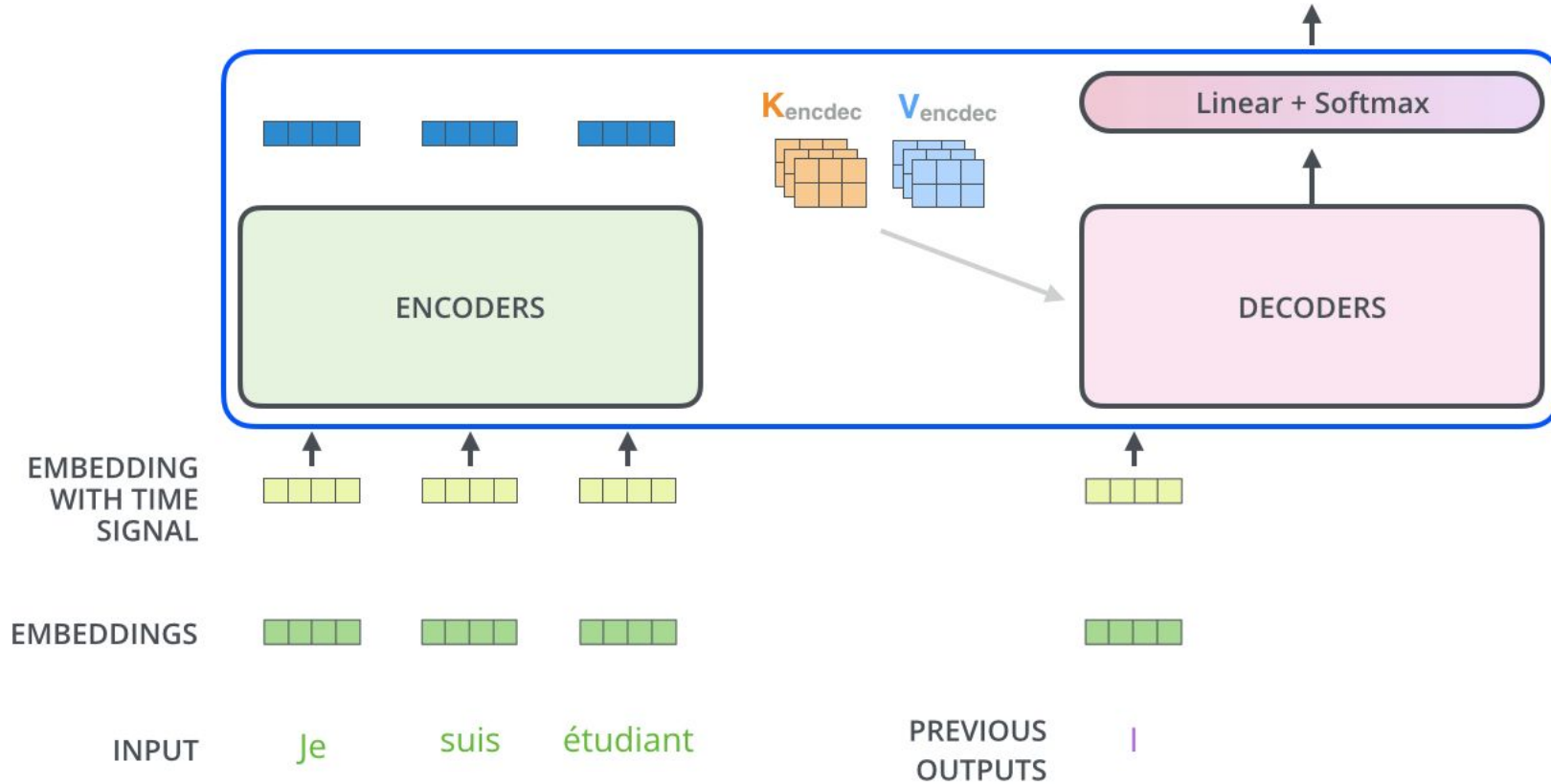


Decoder



Decoding time step: 1 2 3 4 5 6

OUTPUT |



That's All, Thank you!

References

<https://www.saama.com/attention-mechanism-benefits-and-applications/>